



ARIZONA MEDICAL

Tempe, AZ

Project Overview

By: Ryan Clark

Project Overview

Arizona Medical Online Portal

Introduction

The Arizona Medical Online Portal is an innovative solution designed to simplify and enhance the interactions between healthcare providers and patients. This comprehensive digital platform allows for seamless access to medical services and information, significantly improving the efficiency of healthcare operations and enhancing patient satisfaction.

Key aspects

- **Welcome Screen and User Selection:** Upon entering the portal, users are greeted with a choice between patient and staff login, directing them to the appropriate interface based on their role. This initial segregation ensures that the user experience is tailored from the first interaction.
- **Patient Registration and Login:** Patients can create new accounts or log in using existing credentials. This screen is crucial for capturing and verifying patient information, ensuring that the data entered is secure and private.
- **Staff Selection:** Once logged in as staff, users must select whether they are doctors or nurses. This step further refines the access level, ensuring that sensitive patient information is only accessible to authorized personnel.
- **Patient Dashboard:** After login, patients access a dashboard where they can schedule appointments, view past medical records, or communicate with their healthcare providers. This centralized hub makes it easy for patients to manage all aspects of their healthcare.
- **Staff Dashboard:** Healthcare providers have access to a dashboard where they can update patient records, manage appointments, and review patient histories. This interface supports efficient management of patient care and operational tasks.
- **Security Features:** Each interaction within the portal is protected through end-to-end encryption, and data integrity checks are routinely performed to ensure that the information remains secure and unaltered.

Functional Requirements

Our team has developed a user-centric portal that includes several key functionalities aimed at improving the overall healthcare experience. Key features include a secure login system for both patients and staff, which ensures that all user interactions within the portal are safe and private. Patients can easily create and manage their accounts by providing basic personal information, such as their first name, last name, and date of birth.

Once logged in, patients have access to a range of services including the ability to schedule appointments, view past visit histories, access their medical records, and communicate directly with their healthcare providers through a built-in messaging system. This ensures that patients have all the necessary tools at their disposal to manage their health effectively.

For healthcare staff, including doctors and nurses, the portal provides differentiated access levels to ensure sensitive patient data is handled securely. Staff members can view and update patient information, manage visit records, and respond to patient messages. The design of the staff interface is intuitive, allowing for easy navigation and management of patient data, which is crucial in a fast-paced medical environment.

Non-Functional Requirements

In addition to functional capabilities, the Arizona Medical Online Portal has been designed with a strong focus on non-functional requirements to ensure a robust, reliable, and user-friendly system. The portal is designed to be highly usable, with a clear and intuitive user interface that is accessible on various devices including desktops, tablets, and smartphones. It is optimized for performance, with fast loading times and the capability to handle a high volume of concurrent users without any degradation in performance.

Reliability is a cornerstone of the portal's design, ensuring high availability and minimal downtime. Our system includes regular backups and a robust disaster recovery strategy to safeguard against data loss. Scalability is also a key consideration; the portal is built to accommodate growth in user numbers and data volume without the need for significant redesign.

Security measures are rigorously implemented, including regular updates and patches to protect against vulnerabilities and strong authentication and authorization mechanisms to prevent unauthorized access.

Phase-1 Summary

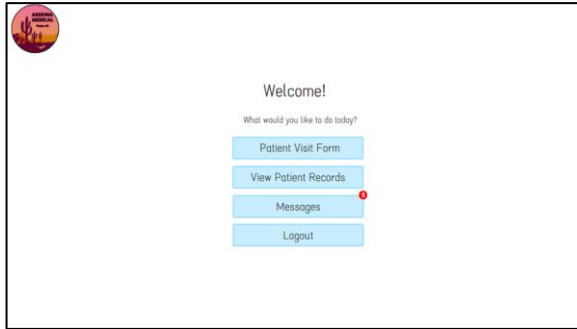
By: Austin Mayhew

UI Walkthrough

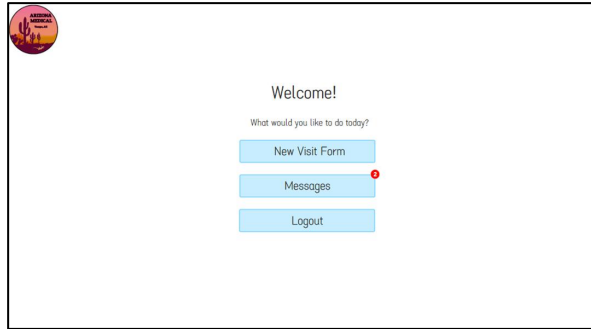
- Identified 15 unique pages
 - 4 login pages
 - 4 patient pages
 - 2 nurse pages
 - 4 doctor pages
 - 1 shared doctor/nurse page

UI Walkthrough

Doctor



Nurse



Patient

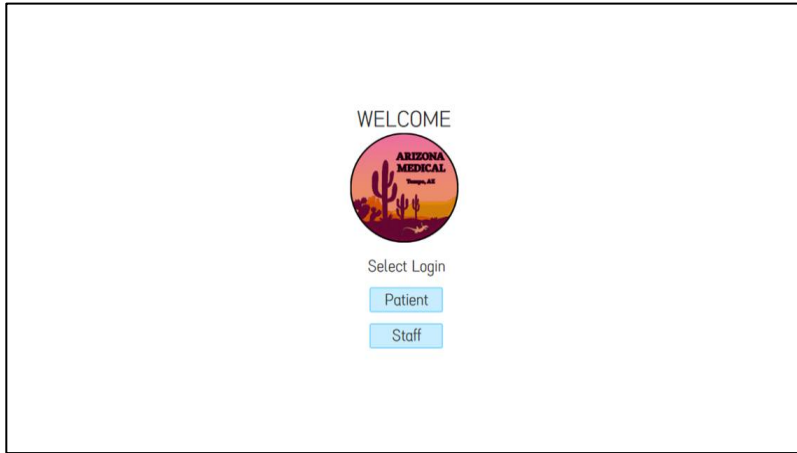


UI Walkthrough

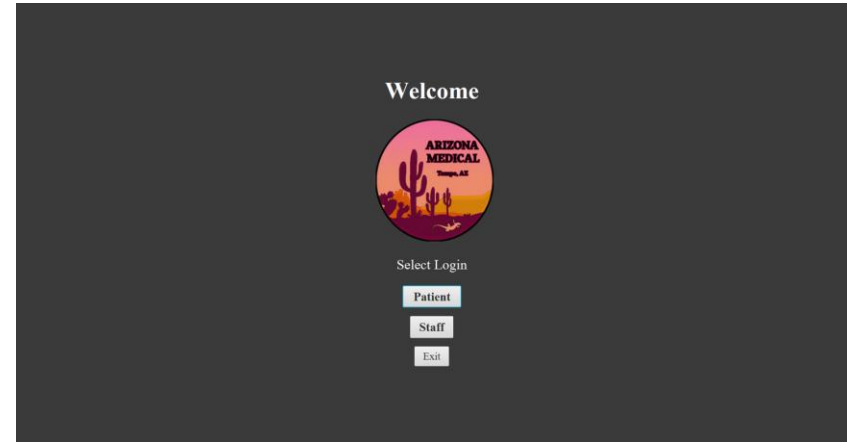
- Simple
- Each element has its own page

UI Walkthrough

Mockup

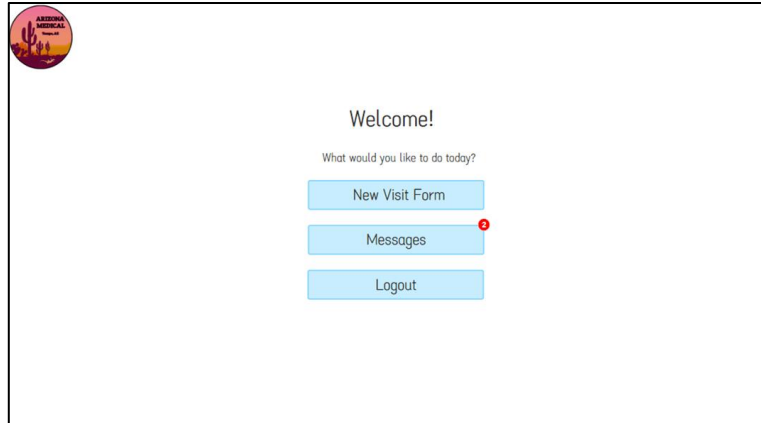


Final Product



UI Walkthrough

- Stayed mostly true to this design
- Some details were cut
- Position of some elements were changed



UI Walkthrough

Mockup



Vitals:

Weight:

<Loaded Data>

 lbs

Height:

<Data>

 "

<Data>

 " feet/inches

Body Temperature:

<Data>

 F°

Blood Pressure:

<Data>

*Patient must be over the age of 12 to enter vitals

Submit for Physical

Exit

Patient Credentials:

First Name:

<Text loaded from relevant .txt file>

Last Name:

<Text loaded from relevant .txt file>

Date of Birth

<Data>

 /

<Data>

 /

<Data>

(MM/DD/YYYY):

Nurse's Notes:

Known Allergies:

<Text loaded from relevant .txt file>

Other Health Concerns:

<Text loaded from relevant .txt file>

Patient's Previous History:

Previously prescribed medications:

<Text loaded from relevant .txt file>

Immunization History

<Text loaded from relevant .txt file>

Previous Health Issues:

<Text loaded from relevant .txt file>

Final Product

Patient Credentials:

Select Patient:

First Name:

Last Name:

Date of Birth:

Visit Date:

dd/mm/yyyy

Vitals:

Weight:

lbs

Height:

<ft>

<ins>

 "

Body Temperature:

F°

Blood Pressure:

*Patient vitals available for age over 12.

Nurse's Notes:

<Known Allergies & Other Health Concerns>

Patient's Previous History:

<Previously Prescribed Medications>

<Immunization History>

<Previous Health Issues>

Submit for Physical

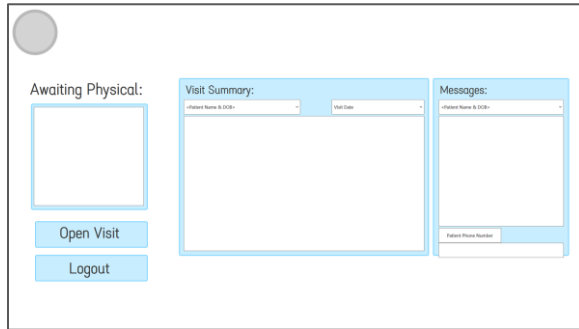
Exit

Alternative Layouts

- Condensed into just 10 pages
- Visit summaries and messages available from menu

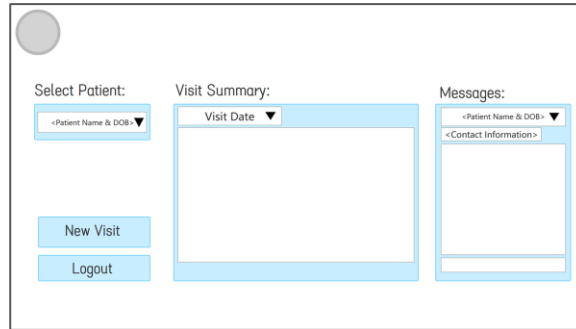
Alternative Layouts

Doctor



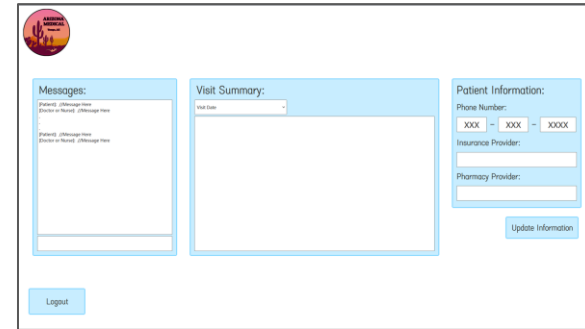
Doctor layout mockup. The interface includes a circular profile icon in the top left. The main content area is divided into three sections: 'Awaiting Physical:' with a large empty box and 'Open Visit'/'Logout' buttons; 'Visit Summary:' with a dropdown for '<Patient Name & DOB>' and a 'Visit Date' field; and 'Messages:' with a dropdown for '<Patient Name & DOB>', a '<Contact Information>' field, and a 'Patient Phone Number' field.

Nurse



Nurse layout mockup. The interface includes a circular profile icon in the top left. The main content area is divided into three sections: 'Select Patient:' with a dropdown for '<Patient Name & DOB>'; 'Visit Summary:' with a 'Visit Date' dropdown; and 'Messages:' with a dropdown for '<Patient Name & DOB>', a '<Contact Information>' field, and a 'Patient Phone Number' field. 'New Visit' and 'Logout' buttons are located at the bottom left.

Patient



Patient layout mockup. The interface includes a circular profile icon in the top left. The main content area is divided into four sections: 'Messages:' with a dropdown for '<Patient Name & DOB>' and a '<Contact Information>' field; 'Visit Summary:' with a 'Visit Date' dropdown; 'Patient Information:' with fields for 'Phone Number' (format XXX-XXX-XXXX), 'Insurance Provider', and 'Pharmacy Provider'; and an 'Update Information' button. A 'Logout' button is located at the bottom left.

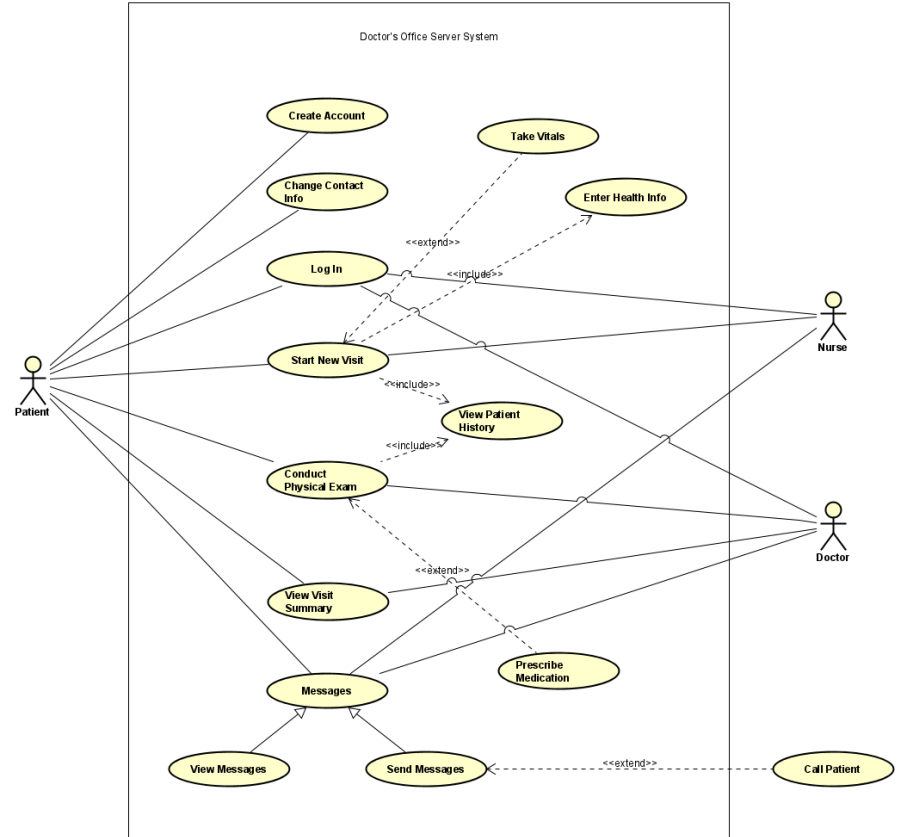
Alternative Layouts

Some problems:

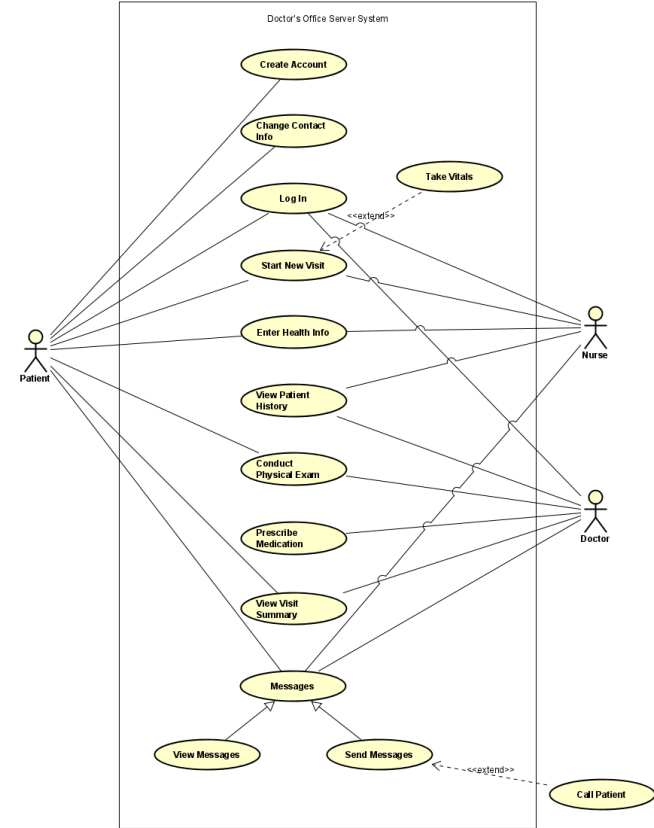
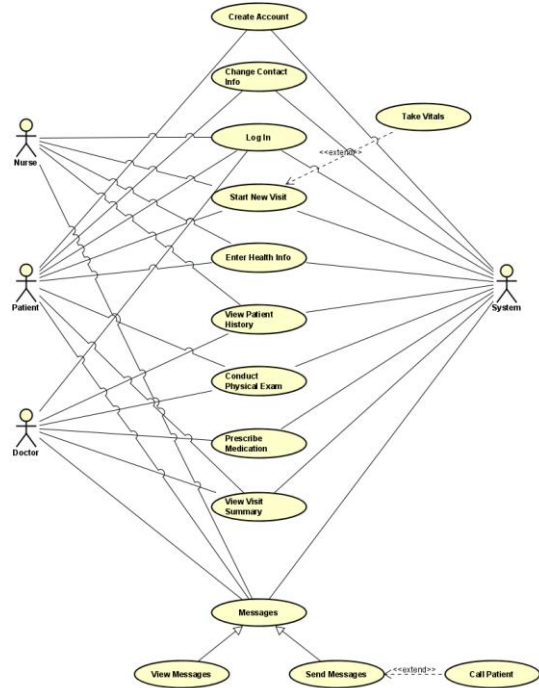
- More complex
- More difficult to implement
- Too late

Use Case Diagram

- Identified 14 unique use cases
- All use case relationship types present
 - 14 association relationships
 - 3 extends relationships
 - 3 includes relationships
 - 1 generalization



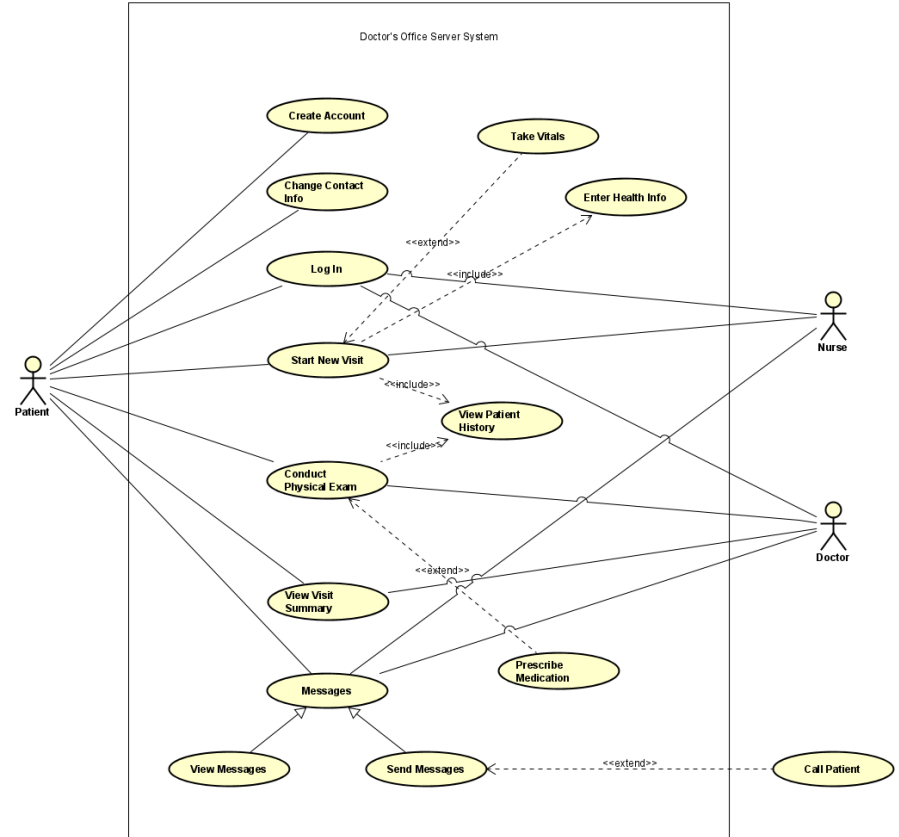
Use Case Diagram



Use Case Diagram

Step-by-step breakdown of a typical visit

- Patient creates account
- Nurse logs in
- Nurse starts new visit
- Doctor logs in
- Doctor conducts exam
- Patient logs in
- Patient views visit summary
- Patient sends message to doctor
- Doctor logs in
- Doctor sees message
- Doctor responds to message



Phase-2 Summary

By: Tristan Andrade

CRC Diagrams

Class: PatientPortal	Uses the system to view their exam results and history. They register themselves into the system, attend physical exams, and discuss prescription plans with doctors and nurses.
Responsibility:	Collaborator:
Create Account	Portals
Change Contact Info	Portals
Log In	Portals
View Visit Summary	Portals
View Messages	Portals
Send Messages	Portals, NursePortal, DoctorPortal
Call Office (Call Patient)	Portals

CRC Diagrams

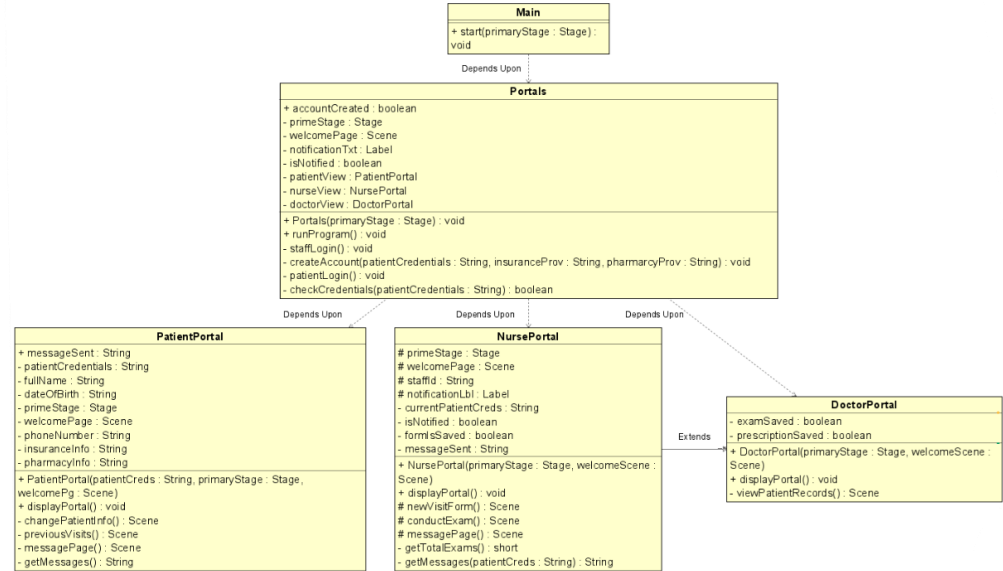
Class: NursePortal	Takes the patient's vitals, current information, and notes any key information such as allergens and new health concerns. They use the system to submit this information for a physical exam and can converse with patients via the system's messaging forum.
Responsibility:	Collaborator:
Log In	Portals
Start New Visit	Portals, PatientPortal
Take Vitals	Portals, PatientPortal
Enter Health Info	Portals, PatientPortal
View Patient History	Portals, PatientPortal?
View Messages	Portals
Send Messages	Portals, PatientPortal, DoctorPortal
Call Patient	Portals, PatientPortal

CRC Diagrams

Class: Doctor	Conducts physical exams and evaluates the exam results as well as the patient's history to determine their best course of treatment. They use the system to view patient history/nurse's notes and assign prescriptions to patients. Like nurses, they also converse with patients to discuss treatment plans through the messaging forum.
Responsibility:	Collaborator:
Log In	Portals
Conduct Physical Exam	Portals, PatientPortal
View Patient History	Portals, PatientPortal?
Prescribe Medication	Portals
View Visit Summary	Portals
View Messages	Portals
Send Messages	Portals, PatientPortal, NursePortal
Call Patient	Portals, PatientPortal

Class Diagram

- The main UI system depends on the portals class
- The portals class depends on the actors
- The portals class bridges the interactive UI with the system's internal code



Phase-3 Summary

By: John Bostater

Program as a whole

The Project code is made up of 5 Objects/.java files.

[Main.java]

[Portals.java]

[PatientPortal.java]

[NursePortal.java]

[DoctorPortal.java]

 azMedical	6/14/2024 12:51 AM	PNG File	68 KB
 DoctorPortal	7/6/2024 10:19 PM	Java Source File	33 KB
 Main	7/6/2024 10:45 PM	Java Source File	3 KB
 NursePortal	7/6/2024 10:18 PM	Java Source File	68 KB
 PatientPortal	7/8/2024 1:01 PM	Java Source File	54 KB
 Portals	7/6/2024 10:20 PM	Java Source File	42 KB

Main.java

Driver for the entire program, it depends upon the Class 'Portals' to function

```
//Relevant JavaFX imports for our GUI
//-----
import javafx.application.Application;
import javafx.stage.Stage;
//-----

//Driver of program: "Arizona Medical"
public class Main extends Application{
    //Driver for our 'Main' Class/Object
    public static void main(String[] args) {
        | launch(args);
    }

    //Sets up the GUI, along with the event handling & functionality of the p
    public void start(Stage primaryStage) {
        //Program Loop:
        //-----
        //Title of the program
        | primaryStage.setTitle("Arizona Medical");

        //Load the Welcome Page of the program and continue from there...
        //We pass primaryStage into the Portal Object to be used within the
        //for seamless page transitions as well as displaying our GUI
        | Portals driver = new Portals(primaryStage);

        //Call upon the public method to run the program
        | driver.runProgram();
        //-----
    }
}
```


Portals.java

Allows access to the <UserType>Portal Objects within the system,

[PatientPortal] [NursePortal]

[DoctorPortal]

```
//Portal Objects [Note]: this is so t
private PatientPortal patientView;
private NursePortal nurseView;
private DoctorPortal doctorView;

//Starts the Program at the Welcome Page
public void runProgram() {
    //From the Welcome Page the user can navigate to the '
    //The 'Login Page' will take the User to the respectiv
    //If the user is to Logout

    //Update all of the other portal objects to null
    //This will [deconstruct our old objects]
    patientView = null;
    nurseView = null;
    doctorView = null;

    //Set the Scene of the Stage & Display it
    primeStage.setScene(welcomePage);
    primeStage.show();
}
```



Portals.java

```
//Create a new Account
newPatient.setOnAction(e -> {
    //Make the user text entry box bigger
    credContainer.setPrefSize(600,375);
    credContainer.setMinSize(600,375);
    credContainer.setMaxSize(600,375);
```

Patient Login

First Name:

Last Name:

Date of Birth: / /
(MM / DD / YYYY)

Phone Number: - -

Insurance Provider:

Pharmacy Provider:

Confirm

Go back

```
//Login --> [Patient]
private void patientLogin(){
```

Patient Login

First Name:

Last Name:

Date of Birth: / /
(MM / DD / YYYY)

Log in

New Patient?

Create Account

Exit

```
//Login --> [Staff]
private void staffLogin(){
```

Select User

Nurse

Doctor

Go Back

PatientPortal.java

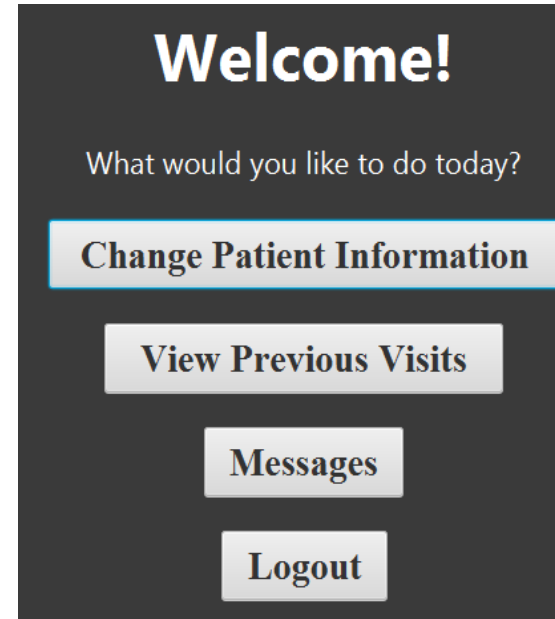
Displays Page for the Patient to interact with and use functionality

```
//Creates & Displays the Patient Portal Scene  
//[NEW NOTE: this method will be used for every  
public void displayPortal() {  
    //Debug Print
```

```
//Change Patient Information {Scene}  
private Scene changePatientInfo(){
```

```
//View Previous Visits {Scene}  
private Scene previousVisits(){  
    //TODO: Implement this method
```

```
//Messages {Scene}  
//You can exit this method/scene  
private Scene messagePage(){  
    //TODO: Implement this method
```



NursePortal.java

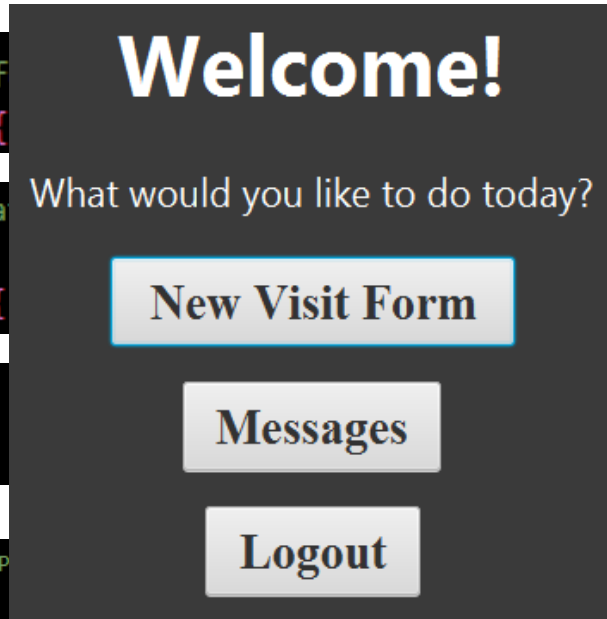
Displays Page for the Nurse to interact with and use functionality.

```
//Run the Main Portal page f
public void displayPortal(){

//[This method looks like the pa
//
protected Scene messagePage(){

//New Visit Form
protected Scene newVisitForm(){

//Since the Doctor portal extends the NurseP
protected String staffId;
```



```
//[DoctorPortal Methods] //return Null in Nurse Portal
//-----
//Empty method that will be Overrided by the DoctorPortal!!
protected Scene conductExam(String patientCredentials){
    //Return null, as the Nurse will NOT use this method
    return null;
}
//-----
```

```
//Object/Class, sub-class of NursePortal  
public class DoctorPortal extends NursePortal{
```

DoctorPortal.java

'extends' NursePortal

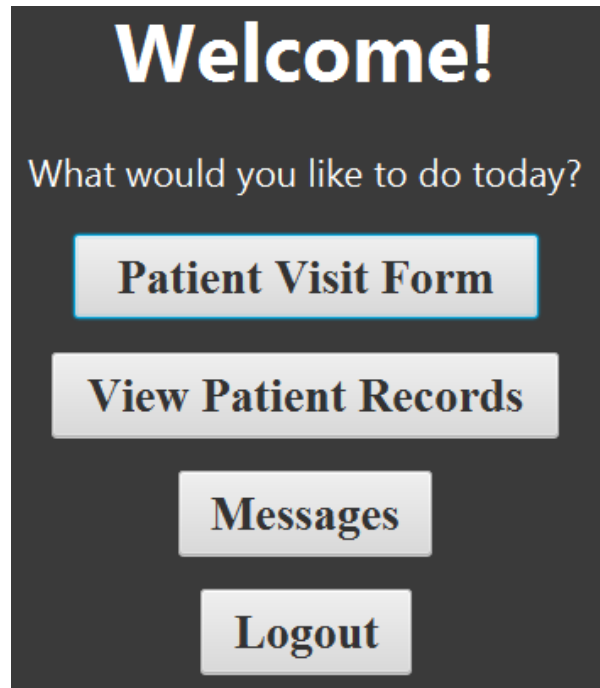
Displays Page for the Doctor to interact with and use functionality.

```
//Display the Main Page  
@Override  
public void displayPortal(){  
    //View Patient Records  
    private Scene viewPatientRecords(){
```

```
@Override  
protected Scene conductExam(String patientCredentials){  
    //New Visit Form
```

Extended (unchanged) methods from Nurse Portal:

```
protected Scene messagePage(){  
    //New Visit Form  
protected Scene newVisitForm(){
```



Conclusions

By: Juan Rosas

Issues Encountered

- Scheduling conflicts with team members
- Data trimming/enhancement discussions
- Complex coding
- Catching up with progress

Project Summary

- Each phase led to a working program
- At least 9 bugs identified (Development Process)
 - 1 remained unsolved (non-critical)
 - NULL Pointer Exception (View Patient Visits: Doctor)
- Around 988 lines of code per object file
- Final Program Size — 266.3 KB

Lessons Learned

- Scrum practices helped with organization
 - Communication is key
- Simple code is better
- Data structures are useful
- Stepping up for others